

In this guide, learn in detail about Stale Element Reference Exception in Selenium, covering the causes and the ways to manage it.

What is Stale Element Reference Exception?#

Exception is the fault or disruption that occurs during the execution of the program at runtime. While testing web applications with Selenium different types of exceptions are faced commonly. These exceptions should be handled carefully to maintain the normal application flow.

StaleElementReferenceException is one of the most common Selenium exceptions which is often surfaced when the web element you are trying to interact with is no longer associated with an HTML element in the DOM. As Selenium tries to **reference a stale element**, it throws a **StaleElementReferenceException**. It is one of the many subclasses of the **WebDriverException** class in Selenium.

Read More: [Exception Handling in Selenium WebDriver](#)

The possible reasons for the web element getting stale can be:

- a page refresh,
- DOM update, or
- location of the web element being changed.

However, web elements do not get relocated automatically, the driver creates a reference ID for the element and has a particular place in DOM that it expects to find.

If the Selenium web driver cannot find the element in the current DOM, any action performed on that web element will throw a **StaleElementReferenceException**.

How to Identify Stale Element Reference Exception in Selenium#

It is possible to identify a Stale Element Reference Exception by analyzing the following:

1. **Exception Message:** The error message will specifically have `StaleElementReferenceException` mentioned, indicating that the element is no longer attached to the DOM.

Example: “stale element reference: element is not attached to the page document”

2. **Error Stack Trace:** The stack trace will point out the line of code where the element reference becomes invalid.
3. **Symptoms:** When interacting with a previously located element (e.g., clicking), the test will throw this exception if the element has been detached or modified.

Example:

```
try {
```

```

WebElement element = driver.findElement(By.id("elementId"));

element.click();

} catch (StaleElementReferenceException e) {

    System.out.println("The element is no longer attached to the DOM.");

}

```

This way, you can spot the exception when it occurs and handle it.

Read More: [How to get Selenium to wait for a page to load](#)

When does Stale Element Reference Exception occur: The Causes#

If you know how Selenium works you must be aware that it tries to find a web element on the web page using [findElement\(\) method](#) of WebDriver. It stores the reference id of that element in memory after the element is found. So, the next time when Selenium wants to interact with the same element, it uses the reference id instead of finding the element again.

The two most usual reasons for the **StaleElementReferenceException** to occur are that:

- The selected web element to be interacted with is no longer present on the HTML web page
- The selected web element was destroyed completely and recreated.

1. The HTML web element is No Longer present on the webpage

There can be scenarios when the web page gets refreshed or updated due to a JavaScript operation and hence the reference of the elements to be interacted with gets stale leading to **StaleElementReferenceException**.

Let us understand with a real time example.

Imagine you are automating a web application which involves filling out a registration form in which clicking on a language dropdown list dynamically changes the content of the country dropdown list.

Now, let's say your Selenium script locates the country dropdown element successfully and selects the country. However, just before the selection is chosen, the user changes the option from the language dropdown and therefore the country dropdown list gets updated dynamically.

At this moment when the Selenium script tries to interact with the country dropdown, it encounters a **StaleElementReferenceException** because the previously located dropdown element is no longer present in the DOM.

2. Permanent deletion of the Referenced Element:

There can be scenarios where the referenced element might have been permanently deleted. For instance, if the web page where the element resides has been refreshed before the interaction with the element actually took place.

Let us understand with a real time example.

Imagine you are automating an e-commerce website which involves navigating to the product, adding it to the cart and later verifying the availability of the product in the cart page.

You create a Selenium script for this scenario, however, let us say that between the time the script clicks the “**Add To Cart**” button and navigates to the cart page, the item gets out of stock. At this moment when the Selenium script tries to verify the presence of the added item on the cart page, it encounters a `StaleElementReferenceException`.

The reference to the element in the cart becomes stale because the item has been permanently removed from the cart due to out of stock.

3. Page Reload or Navigation

The [DOM](#) is rebuilt entirely as the page reloads or navigates to a new URL. Then, any element previously located by Selenium becomes stale as it no longer points to the existing DOM structure.

4. DOM Updates via JavaScript or Ajax

Countless websites utilize Ajax or JavaScript to dynamically update content without having to reload the page. These updates can modify, replace, or remove elements. Thus, invalidating earlier references.

5. Element Re-rendering

[React](#), [Vue](#) or Angular etc., frequently update the UI by re-rendering components. During this process, old elements are replaced with new ones, leading to Selenium losing valid references.

6. Switching Between Frames or Windows

When a script switches from one frame or browser window to another, elements from the previous context become inaccessible. Interacting with them throws a stale element exception.

Read More: [How to test React Apps](#)

How to handle Stale Element Reference Exception in Selenium#

The best way to avoid a `StaleElementReferenceException` is to refresh the reference of a stale element. This means you need to create the element again by applying the locator strategy.

1. Use WebDriverWait

You can use Selenium WebDriver’s explicit wait method to handle `StaleElementReferenceException` by waiting for the element to be **visible**, **clickable**, or any other certain condition.

Instead of using implicit wait, use explicit wait as it allows you to wait for a certain condition to occur before performing any event on the element.

Read More: [Selenium Wait Commands: Implicit, Explicit, and Fluent Wait](#)

There can be following two ways to achieve it:

Method 1: Wait until the element is present

You can use the Explicit wait method to wait for the element to be available before performing any action on it.

```
wait.until(ExpectedConditions.presenceOfElementLocated(Web element locator")))
```

The above line tells the WebDriver to wait until a condition is met for the element such as element to be visible or interactable.

```
public class ExplicitWait {
```

```
public static void main(String[] args) {
```

```
// Initialize WebDriver
```

```
WebDriver driver = new ChromeDriver();
```

```
// Open the webpage
```

```
driver.get("https://google.com");
```

```
// Create an explicit wait
```

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

// Use the explicit wait to handle StaleElementException

try {

    // Wait until the element is present on the page

    WebElement ele = wait

        .until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("textarea[name='q']"))));

    // Perform the action on the element

    ele.sendKeys("Selenium");

    System.out.println("Search text entered successfully.");

} catch (org.openqa.selenium.StaleElementReferenceException e) {

    // Element became stale, handle accordingly

    System.out.println("StaleElementException occurred. Refreshing the page.");
```

```
// Refresh the page
```

```
// driver.navigate().refresh();
```

```
// Re-locate the element after page refresh
```

```
WebElement refreshedEle = driver.findElement(By.cssSelector("textarea[name='q']"));
```

```
refreshedEle.sendKeys("Selenium");
```

```
System.out.println("Search text entered successfully after page refresh.");
```

```
}
```

```
// Close the browser
```

```
driver.quit();
```

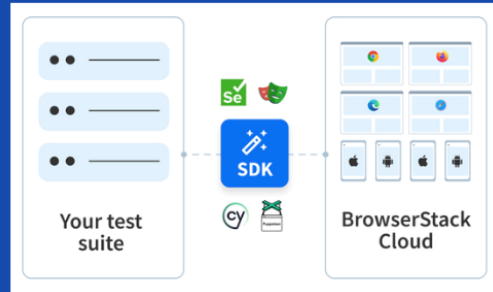
```
}
```

```
}
```

Test Automation on Real Devices

Check out BrowserStack Automate Tool to run your Test Automation Suites on a real device cloud and get comprehensive test results with artifacts.

[Contact Sales](#)



Method 2: Wait until the element is refreshed

You can use **ExpectedConditions.refreshed()** method to overcome the **StaleElementReferenceException** as this method will make the WebDriver wait for the element to be refreshed and take its reference.

```
WebElement ele =  
wait.until(ExpectedConditions.refreshed(ExpectedConditions.presenceOfElementLocated(By.css  
Selector("textarea[name='q']"))));
```

2. Use the try-catch block

The other way to handle this exception is to use a **try-catch** block. The element which is suspected to throw the **StaleElementReferenceException** should be kept under try block and in catch block the web page should be refreshed and the element should be recreated again.

```
WebElement element = driver.findElement(By.id("web element locator"));
```

```
try {
```

```
    element.click();
```

```
} catch (StaleElementReferenceException e) {
```

```
    // Refresh the page
```

```
    driver.navigate().refresh();
```

```
// Try to locate the element again
```

```
element = driver.findElement(By.id("web element locator "));
```

```
element.click();
```

```
}
```

And in some instances, the element is temporarily not interactable through the DOM. In such cases, you can try to access the element for a certain number of times using a loop until it becomes interactable.

```
WebElement ele = driver.findElement(By.cssSelector("textarea[name='q']"));
```

```
for (int i = 0; i < 3; i++) {
```

```
    try {
```

```
        ele.click();
```

```
        break;
```

```
    } catch (StaleElementReferenceException e) {
```

```
        e.printStackTrace();
```

```
    }
```

```
}
```

Here, the loop is expected to run 3 times. If the element is found in the first few attempts it will break the loop and come out. Otherwise, it will keep finding it till the loop ends.

3. Use Page Object Model

In POM [Design Pattern](#), Page Factory is an inbuilt POM concept used for initializing page objects. In this pattern, using **@FindBy** annotation to locate an element. It helps to update the reference of the web element each time before any action is performed on it.

Page Factory initializes the page elements lazily which means they are initialized only when they are accessed within page methods.

If the elements become stale, it will automatically reinitialize it whenever it is accessed the next time.

Read More: [Page Object Model and Page Factory in Selenium](#)

Below is a sample program to understand the Page factory structure.

```
public class LoginPage {
```

```
    private WebDriver driver;
```

```
    @FindBy(id = "username")
```

```
    private WebElement usernameInput;
```

```
    @FindBy(id = "password")
```

```
    private WebElement passwordInput;
```

```
    @FindBy(id = "loginButton")
```

```
private WebElement loginButton;
```

```
// Constructor to initialize WebDriver and elements
```

```
public LoginPage(WebDriver driver) {
```

```
    this.driver = driver;
```

```
    PageFactory.initElements(driver, this);
```

```
}
```

```
// Method to perform login action
```

```
public void login(String username, String password) {
```

```
    usernameInput.sendKeys(username);
```

```
    passwordInput.sendKeys(password);
```

```
    loginButton.click();
```

```
}
```

}

4. Refresh the web page

Refreshing a web page can sometimes aid in avoiding

StaleElementReferenceException, because when Selenium refreshes the web page, the browser reloads the entire DOM which helps in reinitializing the elements. So, if for any reason the element gets stale, refreshing the web page can make the element accessible again.

However, it is not a reliable solution and should be used very wisely. Refreshing the web page will rebuild the DOM completely which may lead to performance issues and other complexities.

Note: This should be opted as the last solution if none of the above solutions work as it will only temporarily help you to solve the problem.

Why is Testing on Real Devices and Browsers important?#

Different devices and browsers render web pages differently. To ensure that a given application behaves uniformly across different pools of devices and browsers, it is advisable to test it in the real devices and browsers with a real environment set up.

Testing in [real devices](#) and browsers ensures high-quality user experience, compatibility across different platforms, and validates performance and security of the application in [real user conditions](#). BrowserStack is one such platform which leverages testing any mobile or web application by providing 3500+ real devices.

[Talk to an Expert](#)

Conclusion

When a previously located element has become invalid because of changes in the DOM, a stale element reference exception occurs in Selenium. Its causes could include page reloads, element removal, etc. Understanding these causes can help QA teams develop stable, flawless scripts by re-locating elements or using appropriate wait strategies.